

The background image shows a bike-sharing station. In the foreground, several red bicycles are parked in a row on a brick-paved surface. A grey kiosk with the 'bikeshare' logo is visible on the left. In the background, there is a parking lot with several cars and a few people walking. The scene is outdoors during the day.

Bike Sharing Demand Prediction

Angelina Bautista and Molly Scally

Introduction



Image: Flickr

What is a bike-sharing system?

- Public service offering short-term, on-demand bike rentals

System analyzed:

- Capital Bikeshare (Washington, D.C.)

Challenge: demand fluctuates by time, weather, and commuting patterns

- Too few bikes: poor user experience
- Too many bikes: wasted resources

Business Goals

- Predict demand to optimize redistribution of bikes
 - improve bike availability
 - reduce operational costs
 - enhance user experience
- Approach problem by using regression models
 - predict a numeric value
 - use predictive insights to guide business decisions

Dataset Overview

Source: Kaggle Bike Sharing Dataset

- Core data from Capital Bikeshare, aggregated with weather/seasonal data by University of Porto (Portugal)

Structure:

- 17,379 observations x 17 features

Granularity: hourly data (2011-2012), representing total system-wide demand

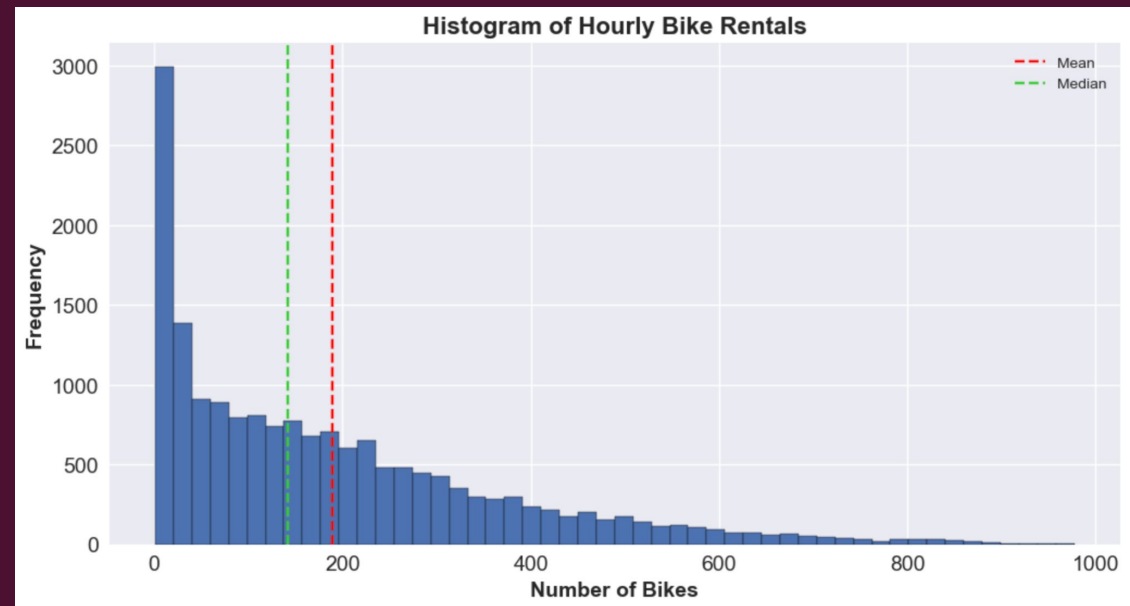
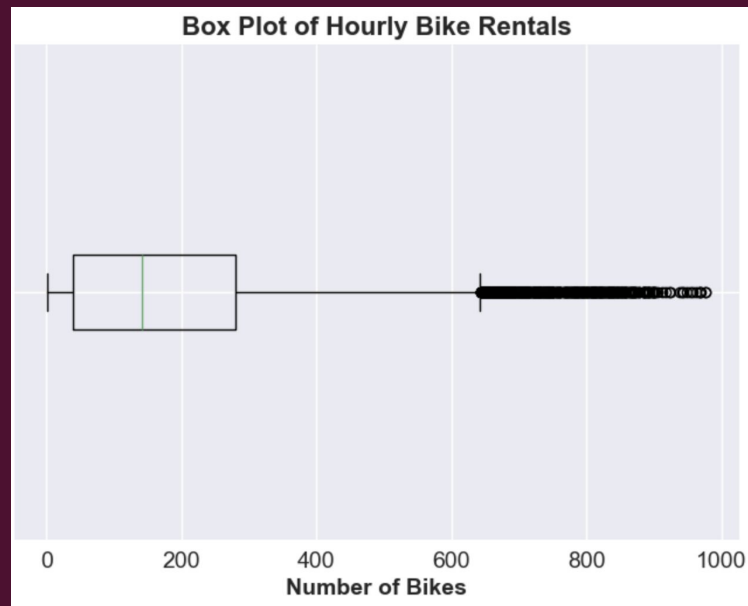
Columns:

- A unique identifier for each hour: ``instant``
- Time-based: ``dteday`` (exact date, in YYYY-MM-DD), ``hr`` (hour), ``weekday``, ``mnth`` (month), ``season``, ``yr`` (year)
- Categorical: ``holiday``, ``workingday``, ``weathersit`` (type of weather, e.g. sunny and clear/cloudy/storms)
- Numerical: ``temp`` (temperature), ``atemp`` (“feels like” temperature), ``humidity``, ``windspeed``, ``casual`` (number of one-off bike users), ``registered`` (number of registered, habitual users), ``cnt`` (total rentals, the target)

Target Distribution

Key Observations

- Right-skewed (many low-demand hours, few very high ones)
- May benefit from log transformation



Key Patterns from EDA

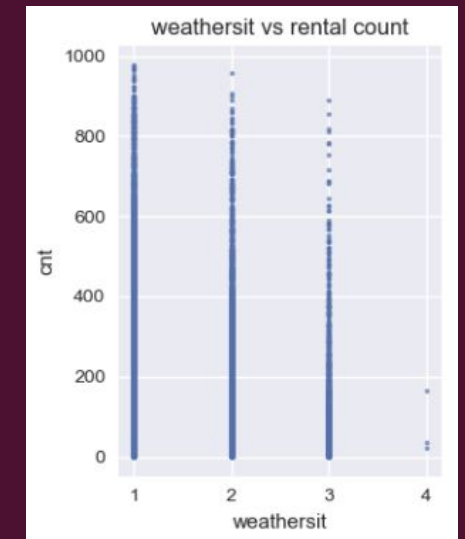
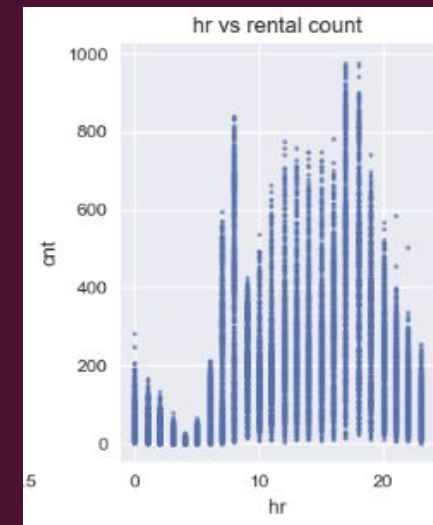
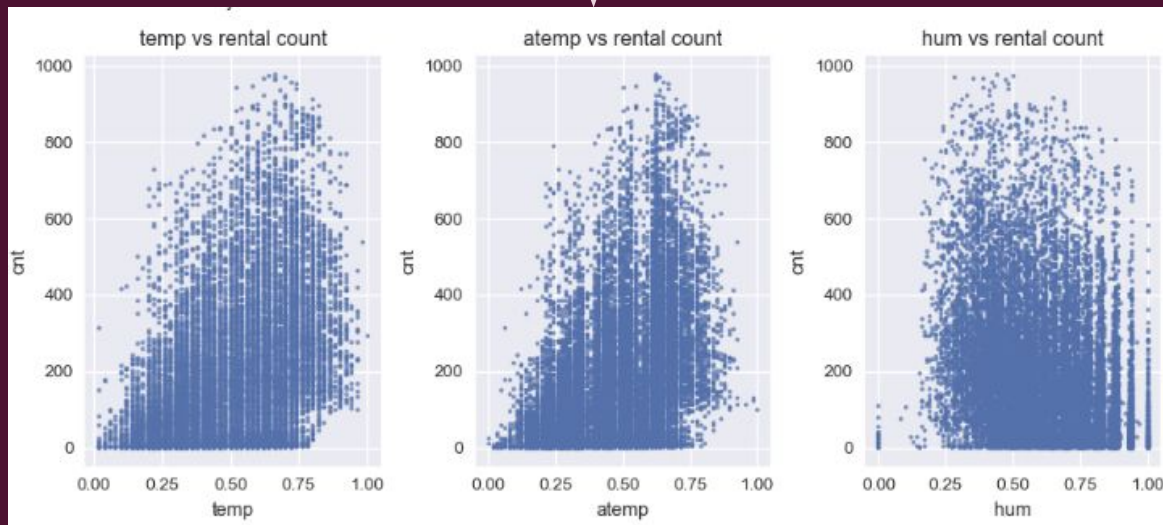
Temperature and rentals appear to rise together

Multicollinearity risk: temp and atemp ($r=0.99$)

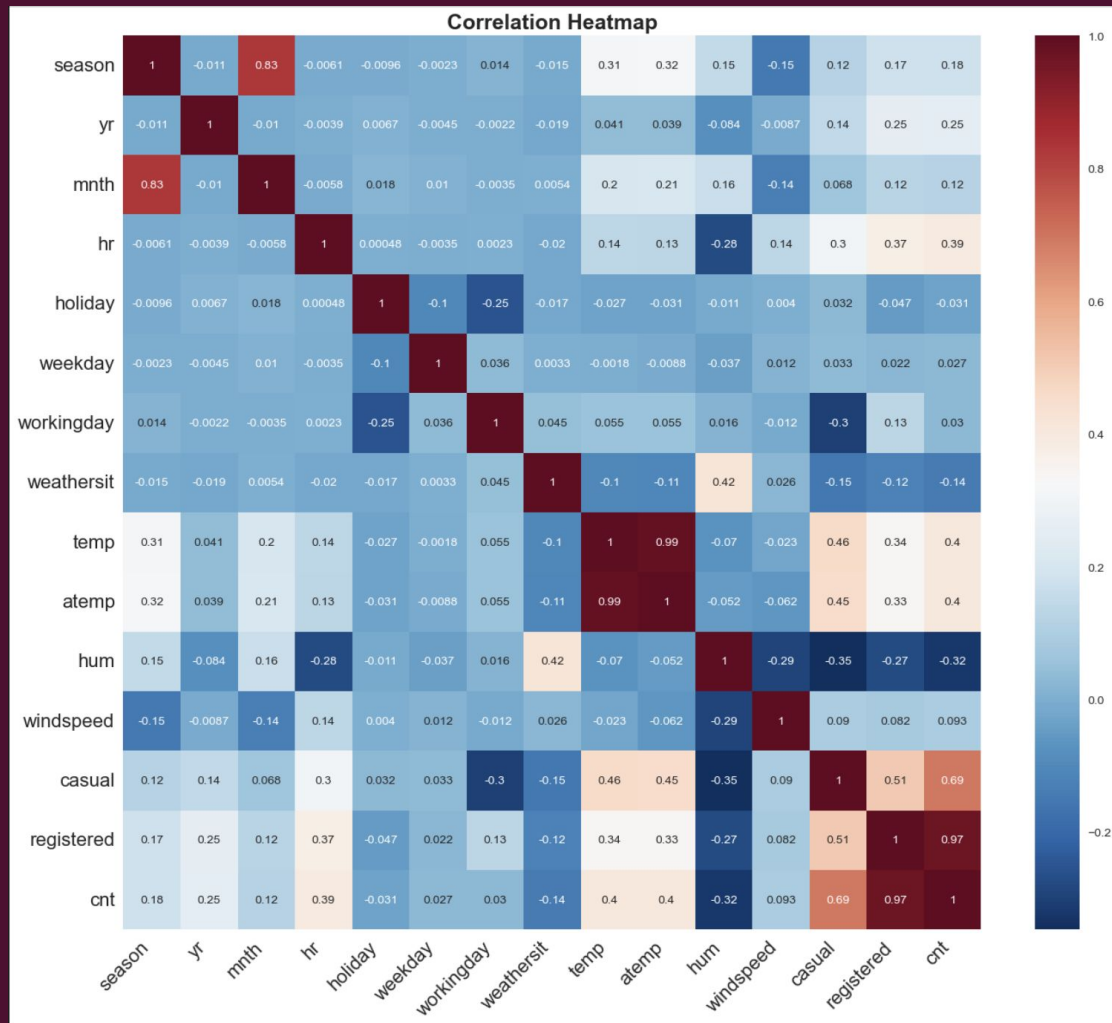
As humidity rises, rentals fall

Hour of the day associated with structural demand patterns

Weather quality has strong impact on demand



Key EDA Patterns: Correlation Heatmap



Registered users have the strongest impact on total bike count ($r=0.97$).

Season and month are strongly related due to redundancy ($r=0.83$).

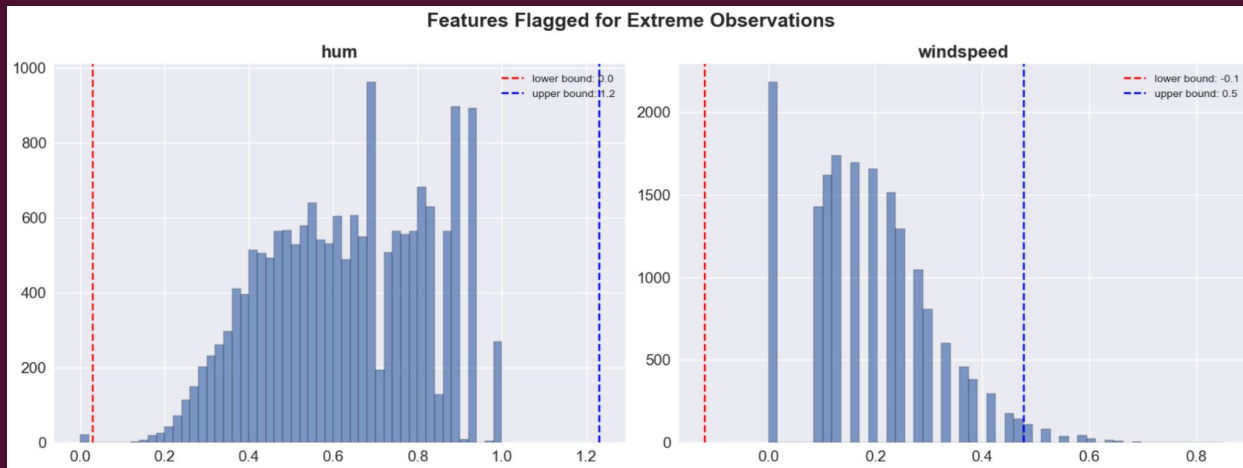
Data Preparation

Key Preprocessing Decisions:

Dropped `instant` and `dteday` – no predictive power

Removed data leakage by dropping `casual` and `registered`, which prevents over inflated model performance

One-hot encoding of time and weather variables, which increased feature count from 17 to 62



Outlier Handling: kept extreme values in `hum` and `windspeed`, which were rare and reflective of real-world conditions

Outliers are rare (<2%), and within realistic ranges

Feature Engineering

New features created:

Time-based:

`is\_dawn`
`is\_morning`
`is\_afternoon`
`is\_evening`
`rush\_hour`

Environmental:

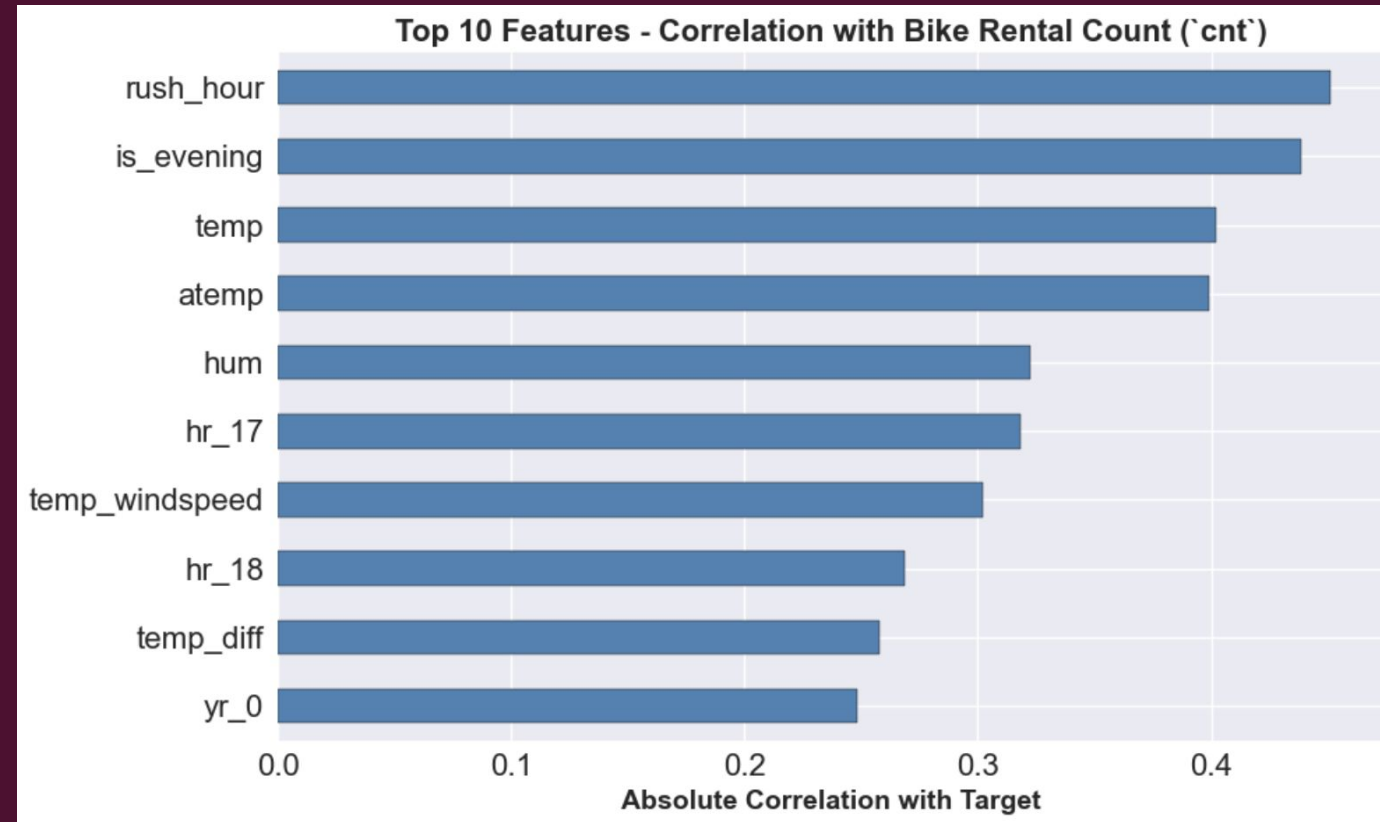
Extreme flags (for temperature, humidity, wind speed)
`temp\_diff` (difference between temperature and “feels like”)

Interactions:

`temp\_windspeed`
`hum\_windspeed`

Finally, a version of the data with standard scaling applied to the numerical variables, for use in several models.

Most useful feature: `rush\_hour` ($r=0.4513$ with demand), captures commuting behavior



Time-based features dominate predictive power

Target and Scaling Decisions

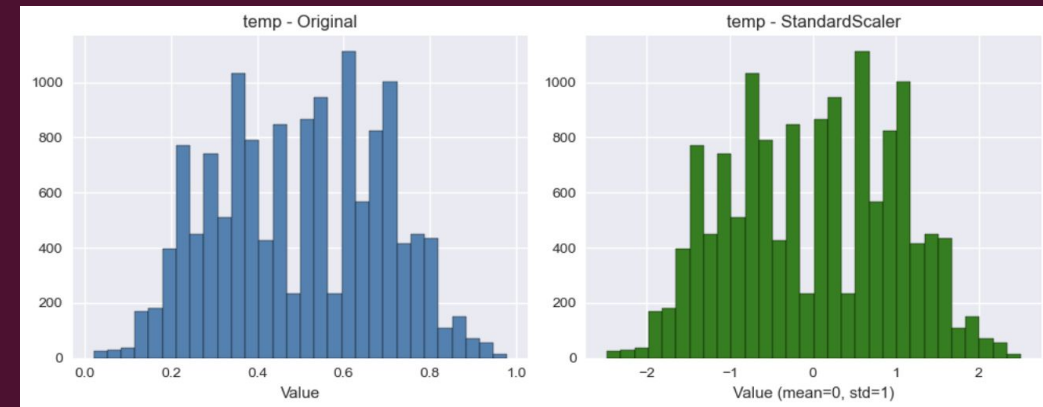
Target (`cnt`) Distribution: log transformation ultimately not applied, allowing preservation of real demand spikes

Outliers: retained, as they represent real-world extremes

Feature Scaling: standardization of numerical features

Performance Impact:

- baseline model achieved $R^2=0.69$, RMSE=100
- Model handled skew/outliers without transformation



Standard scaling centers features (mean = 0) and normalizes variance (std = 1)

Modeling Strategy

Selection Approach

- Compare simple versus complex approaches
- Evaluate linear versus nonlinear relationships
- Take advantage of regularization and feature selection (Ridge, Lasso)

Linear Models:

- Linear Regression
- Ridge
- Lasso

Tree-based Models:

- Decision Tree
- Random Forest
- Gradient Boosting

Evaluation Approach

- Compared baseline, tuned models
- Primary metric: Test R^2 (+ RMSE)
- Evaluated both performance and generalization

All Models: baseline, engineered, and tuned

Type	Model	Data	Train R ²	Test R ²	Δ R ²
Baseline	Linear Regression	Raw data (no engineered features)	0.685	0.689	0.004
		Unscaled	0.713	0.715	0.002
		Scaled	0.713	0.715	0.002
	Ridge	Scaled	0.713	0.715	0.002
	Lasso	Scaled	0.713	0.715	0.002
	Decision Tree	Unscaled	0.928	0.885	-0.044
	Random Forest	Unscaled	0.986	0.941	-0.045
	Gradient Boosting	Unscaled	0.906	0.900	-0.006
Tuned	Ridge	Scaled	0.713	0.715	0.002
	Random Forest	Unscaled	0.992	0.944	-0.048
	Gradient Boosting	Unscaled	0.972	0.941	-0.031

Ridge Regression: stable but underfitting

Type	Model	Data	Train R ²	Test R ²	Δ R ²
Baseline	Linear Regression	Raw data (no engineered features)	0.685	0.689	0.004
		Unscaled	0.713	0.715	0.002
		Scaled	0.713	0.715	0.002
	Ridge	Scaled	0.713	0.715	0.002
	Lasso	Scaled	0.713	0.715	0.002
	Decision Tree	Unscaled	0.928	0.885	-0.044
	Random Forest	Unscaled	0.986	0.941	-0.045
	Gradient Boosting	Unscaled	0.906	0.900	-0.006
Tuned	Ridge	Scaled	0.713	0.715	0.002
	Random Forest	Unscaled	0.992	0.944	-0.048
	Gradient Boosting	Unscaled	0.972	0.941	-0.031

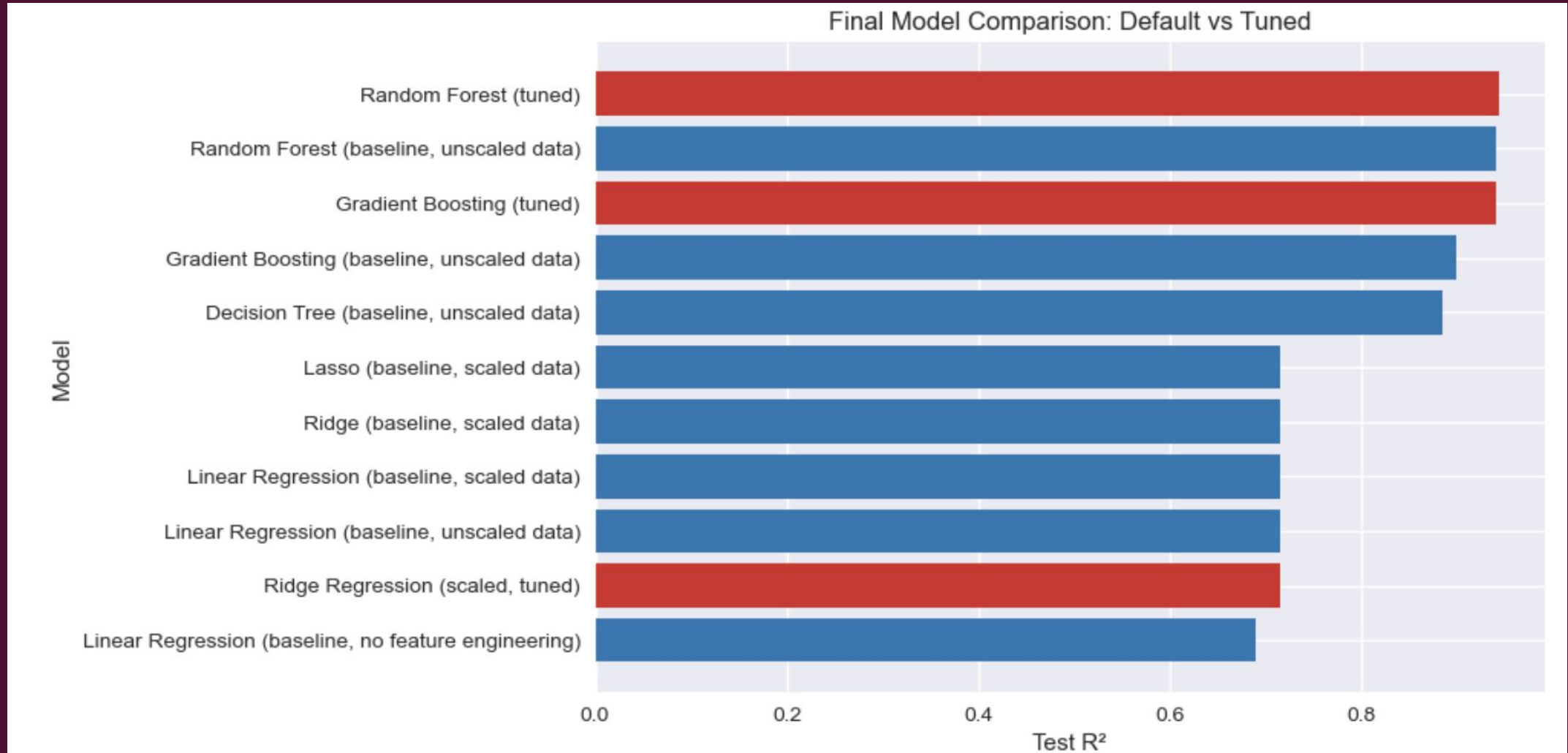
Random Forest: best accuracy, slight overfitting

Type	Model	Data	Train R ²	Test R ²	Δ R ²
Baseline	Linear Regression	Raw data (no engineered features)	0.685	0.689	0.004
		Unscaled	0.713	0.715	0.002
		Scaled	0.713	0.715	0.002
	Ridge	Scaled	0.713	0.715	0.002
	Lasso	Scaled	0.713	0.715	0.002
	Decision Tree	Unscaled	0.928	0.885	-0.044
	Random Forest	Unscaled	0.986	0.941	-0.045
	Gradient Boosting	Unscaled	0.906	0.900	-0.006
Tuned	Ridge	Scaled	0.713	0.715	0.002
	Random Forest	Unscaled	0.992	0.944	-0.048
	Gradient Boosting	Unscaled	0.972	0.941	-0.031

Gradient Boosting: strong performance, best generalization

Type	Model	Data	Train R ²	Test R ²	Δ R ²
Baseline	Linear Regression	Raw data (no engineered features)	0.685	0.689	0.004
		Unscaled	0.713	0.715	0.002
		Scaled	0.713	0.715	0.002
	Ridge	Scaled	0.713	0.715	0.002
	Lasso	Scaled	0.713	0.715	0.002
	Decision Tree	Unscaled	0.928	0.885	-0.044
	Random Forest	Unscaled	0.986	0.941	-0.045
	Gradient Boosting	Unscaled	0.906	0.900	-0.006
Tuned	Ridge	Scaled	0.713	0.715	0.002
	Random Forest	Unscaled	0.992	0.944	-0.048
	Gradient Boosting	Unscaled	0.972	0.941	-0.031

Visual Summary of Models: Capturing nonlinear patterns is key



Hyperparameter Tuning Results

Ridge

Parameter tuned:
regularization strength (α)

Best:
 $\alpha=100$

Random Forest

Parameters tuned:
maximum depth of each tree (max_depth)
number of features considered at each split (max_features)
number of trees (n_estimators)

Best:
max_depth=None,
max_features=0.5,
n_estimators=200

Gradient Boosting

Parameters tuned:
step size for boosting (learning_rate)
maximum depth of each tree (max_depth)
number of trees (n_estimators)

Best:
learning_rate=0.2,
max_depth=5,
n_estimators=200

Model Reliability & Validation

Validation Strategy

5-fold cross validation

Overfitting Analysis

Random forest: slight overfitting ($\Delta R^2 = -0.048$)

Gradient boosting: less overfitting ($\Delta R^2 = -0.031$)

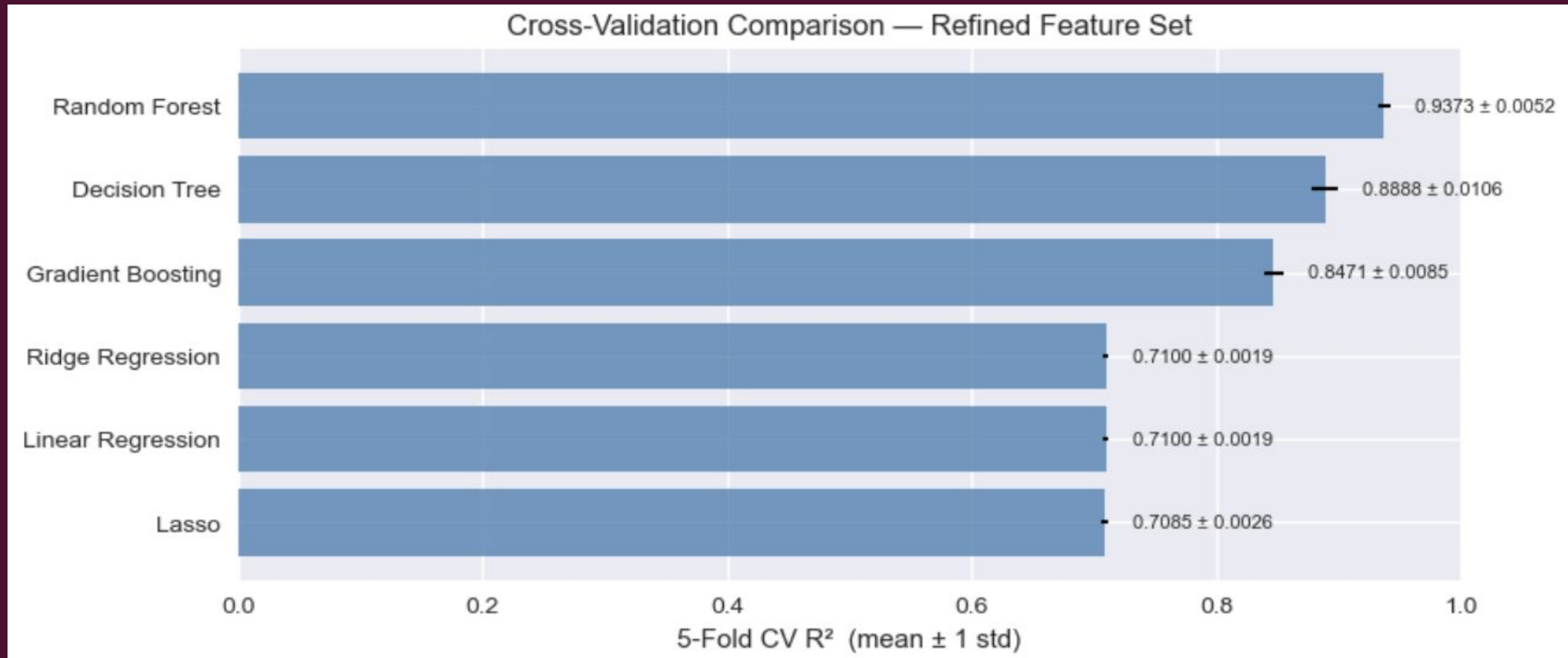
Ridge: underfitting ($\Delta R^2 = 0.002$)

Small gap between CV and test R^2 : confirms that results are not due to a single split

Tuned model	CV R^2 Mean	CV R^2 std	Test R^2	Test RMSE
Random forest	0.9393	0.0058	0.9440	42.7863
Gradient Boosting	0.9375	0.0045	0.9407	44.0388
Ridge Regression	0.7100	0.0018	0.7149	96.5454

Low variance (CV std < 0.01 for top models): consistent performance across splits

CV Results



Cross-validation confirms consistent performance and strong generalization

Final Model & Error Analysis

Best Model:
Tuned Random Forest

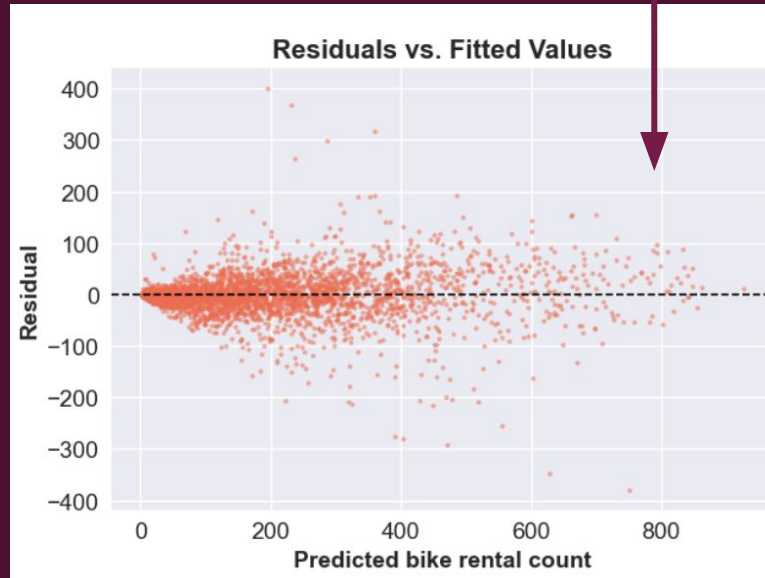
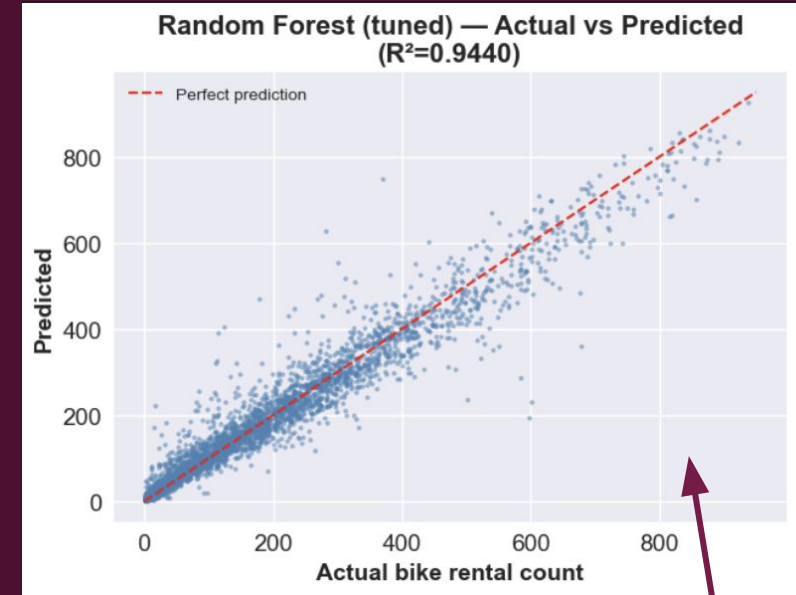
Performance (test set):

- $R^2 = 0.944$
- RMSE=42.8 bikes

What does RMSE mean?

- Predictions are off by ~43 bikes per hour on average

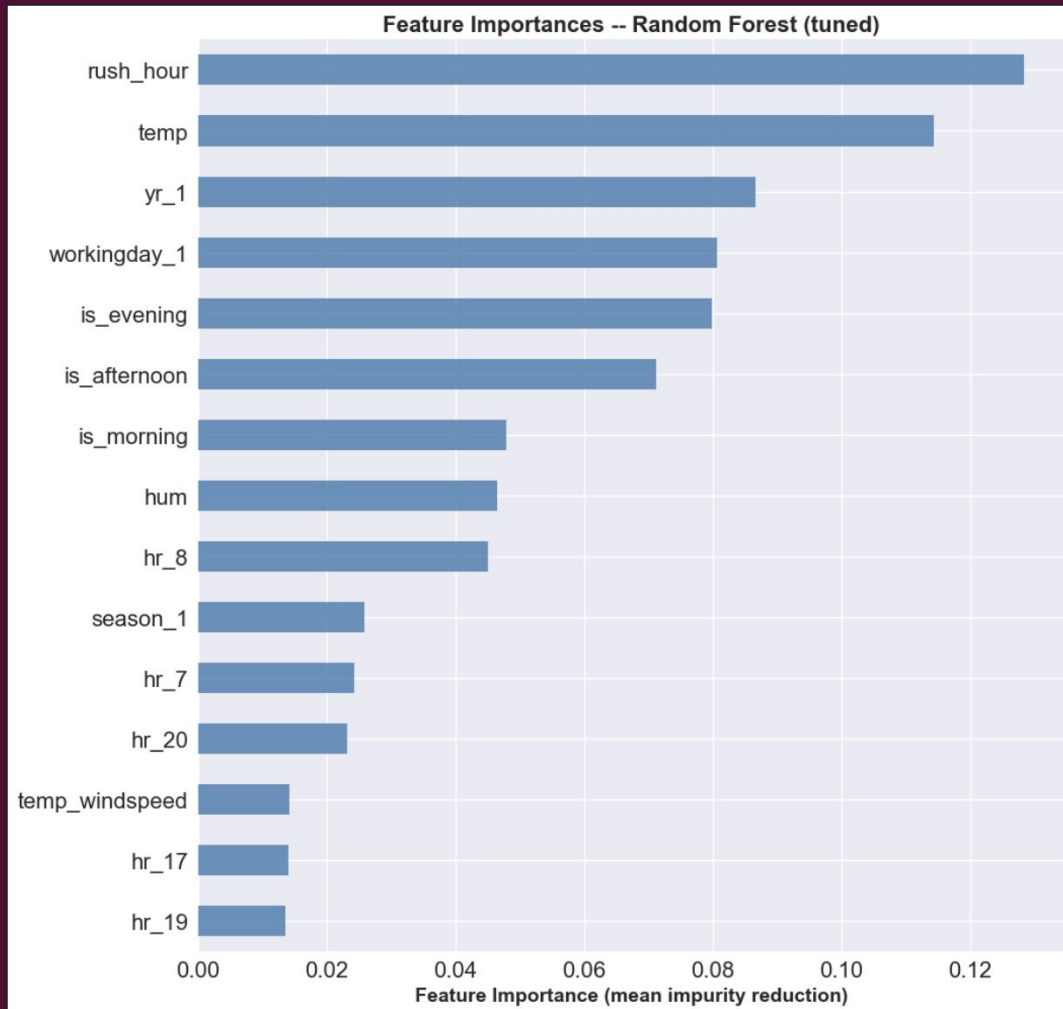
Slight spread at higher demand; means its harder to predict peak usage



Model predictions track actual values closely

Residuals are centered around 0; no major bias

Key Drivers of Bike Demand



Top Features

1. is it between hours 5-7 on a working day? (rush_hour)
2. temperature (temp)
3. year (yr)
4. is the day a working day? (workingday_1)
5. time-of-day indicators (is_evening, is_afternoon, is_morning)

What does this mean?

- demand driven primarily by commuting patterns
- weather (temperature) and time (hour of the day, type of day) are strongly influential
- usage increased over time (year effect)

Conclusion: Actionable Recommendations

1. **Optimize bike distribution for rush hours**
 - Highest demand driven by commuting patterns (`rush\_hour`)
 - Strategically position the bikes before peak times
2. **Adjust planned operations based on the weather forecast**
 - Demand was linked to temperature
 - Scale up supply on warm days, scale down on poor weather days
3. **Build in flexibility for peak demand uncertainty**
 - Higher prediction error during peak hours
 - Maintain extra capacity to prevent shortages, to allow more responsive rebalancing during spikes

Limitations and Next Steps

Key Limitations

1. Size of feature set

- No data on events, location
- May miss factors of extreme demand

2. Temporal aspects not fully leveraged

- Observations were treated independently; did not model sequential/time-series effects

Next Steps

- Incorporate richer data sources, e.g. weather forecasts, event schedules, station-level data, data beyond 2012
- Explore time-series models to capture temporal patterns such as trends, seasonality
- Experiment with target log-transformation to address skew and extreme values and compare impact on model performance
- Further refine feature set to reduce redundancy and improve interpretability and efficiency
- Deploy for real-time forecasting

Thank you!

Questions?

